Collective Robotics – Tutorial 2

Robot Behaviors and Swarm Systems

Hochschule Bonn-Rhein-Sieg
Prof. Dr. Javad Ghofrani

Bhavesh Gandhi Jaqueline Machida

31.05.2026

Contents

1 Task 1 – Behaviors of a Single Robot

1.1 Overview

We implemented three reactive behaviors for a simulated vacuum-cleaner robot in the TurtleBot3 House world: **collision avoidance**, **wall following**, and **vacuum-cleaner coverage** (boustrophedon). The robot model, world, launch files, and ROS 2 nodes all live in the `single_robot` package under `Task 1/ros2_ws/`.

Simulation setup. We used **ROS 2 Jazzy** with **Gazebo Harmonic**. A custom SDF model (`models/vacuum_cleaner/model.sdf`) defines a cylindrical robot (radius 0.17 m, height 0.09 m) with two differential-drive wheels, a passive caster, and a 360° 2D lidar mounted on top. The Gazebo `DiffDrive` system provides the wheel control, and a `ros_gz_bridge` parameter bridge translates `/scan`, `/cmd_vel`, `/odom`, `/tf`, and `/joint_states` between Gazebo and ROS 2.

A snapshot of the simulation environment (TurtleBot3 House) with the vacuum cleaner robot spawned inside it is shown in Figure 1.

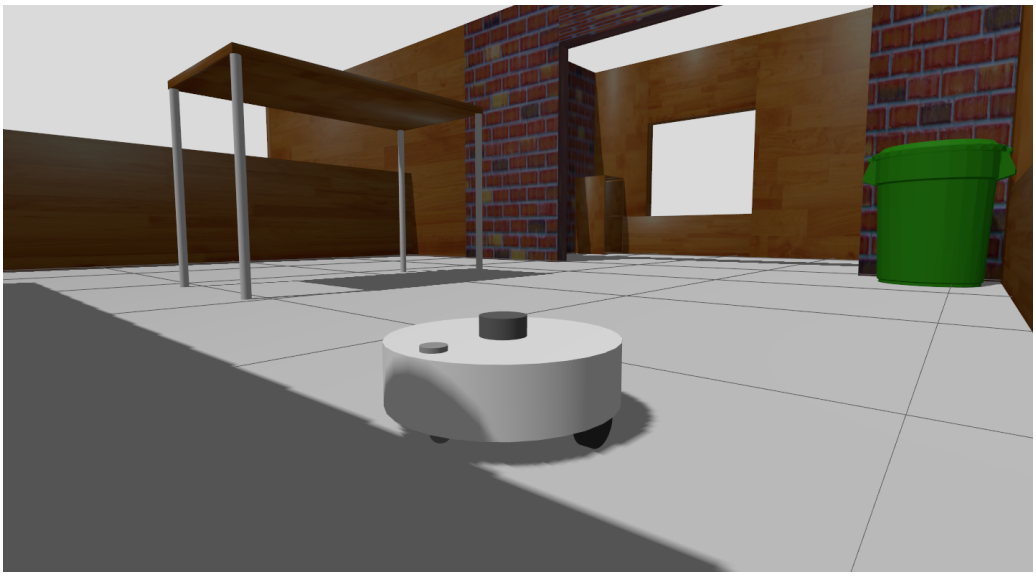

Figure 1: Top-down view of the vacuum-cleaner robot in the TurtleBot3 House world used for all three Task 1 experiments.

Common design. All three behaviors share the same skeleton: a `LaserScan` subscriber caches the latest scan, a timer drives a 20 Hz control loop that walks the scan once per tick to compute sector minima (front cone of ±30, plus left/right halves), updates a finite state machine, and publishes a `Twist` on `/cmd_vel`.

1.2 Collision Avoidance

The robot drives forward at constant speed and rotates in place when an obstacle enters the front cone. The FSM has two states:

- **CRUISE** – publish `linear.x = CRUISE_SPEED`.

- **AVOID** – stop linear, rotate at ±`TURN_SPEED` toward the freer side (chosen at transition time by comparing left/right sector minima).

Hysteresis on the trigger and clear distances (`TRIGGER_DISTANCE = 0.35 m`, `CLEAR_DISTANCE = 0.55 m`) prevents the robot from chattering between the two states near the threshold. The turn direction is frozen for the duration of an AVOID episode so the robot commits to one direction rather than oscillating between left and right.

A demonstration video is available at https://youtu.be/UbriNbjf1nM.

1.3 Wall Following

The robot performs right-wall following using a small FSM and a PD controller on the side-cone distance:

- **FIND_WALL** – drive forward until the right-side cone (a ±30 window centered at $-\pi/2$) sees a wall within `WALL_DETECT_DISTANCE = 0.8 m`.
- **FOLLOW** – maintain a target distance of 0.4 m to the right wall using $\omega = K_P \cdot e + K_D \cdot \dot{e}$, where $e = d_{\text{target}} - d_{\text{right}}$. Positive error (too close) commands a left turn (away from the wall); negative error (too far) commands a right turn (toward the wall).
- **CORNER_TURN** – when the front cone is blocked (concave corner), stop linear and rotate left in place until the front clears. Hysteresis on `CORNER_TRIGGER`/`CORNER_CLEAR` prevents flapping.

Convex corners are handled implicitly: when the right wall disappears, the measurement is clamped to `WALL_DETECT_DISTANCE`, which gives a constant rightward bias that curls the robot back toward the wall. An edge case handled during development was the initial transition: entering FOLLOW from a *front* detection (instead of a side detection) gave the PD controller no usable measurement and caused the robot to spin in place; the fix was to require the wall to appear on the right side before entering FOLLOW, and to redirect head-on detections directly to CORNER_TURN.

A demonstration video is available at https://youtu.be/crJzeYGZqmE.

1.4 Vacuum-Cleaner Coverage

Coverage strategies. Before settling on an implementation we considered three families of strategies for a vacuum-cleaning robot:

1. **Random walk.** Drive straight; on collision, turn by a random angle and continue. The simplest possible strategy, requiring no state beyond the FSM and no odometry. Given enough time it probabilistically covers the entire reachable area, but the time to near-full coverage is long and the trajectory looks erratic.
2. **Lawnmower (boustrophedon).** Drive forward until a wall, U-turn while shifting laterally by one body width, then drive in the opposite direction. The repeating parallel stripes cover the area systematically and quickly. The main challenges are (i) executing crisp 90 turns so successive stripes remain parallel to each other and perpendicular to the walls, and (ii) handling the room boundary: once the robot has marched all the way across the room, the lateral shift gets blocked by the parallel

wall and the robot becomes stuck oscillating in the last stripe. Reversing the shift direction at that point keeps the robot moving but only re-covers area it has already cleaned, and it does not guarantee that gaps left by inaccurate stripe spacing get filled.

3. **Map-based planning.** Build an occupancy map of the environment, track the robot's pose against it, and direct the robot toward un-cleaned cells while restarting the local boustrophedon sweep there. This is what current commercial vacuum robots do. It dominates the other two in efficiency and coverage guarantee, but requires SLAM and a planning layer.

Our implementation. We implemented the boustrophedon strategy, since it directly demonstrates the characteristic "zig-zag" coverage pattern. The FSM has four states:

- **SWEEP** – drive forward at `FORWARD_SPEED` until the front cone reads below `TRIGGER_DISTANCE`.
- **TURN_A** – rotate 90 in the current `turn_direction`, using odometry yaw as the target.
- **SHIFT** – drive forward by one `BODY_WIDTH` (0.34 m) measured against `/odom` position, or until the front cone trips, or until a wall-clock timeout fires.
- **TURN_B** – rotate another 90 in the same direction, which inverts the heading and leaves the robot laterally shifted by one stripe. The `turn_direction` is then flipped so the next U-turn rotates the other way, keeping the lateral shift on the same side of the room and producing parallel stripes.

Turn precision (proportional control). A bang-bang turn at full `TURN_SPEED` produces a large yaw overshoot when commanded to stop, because the diff-drive plugin has finite angular deceleration. After a single U-turn the heading was off by $\sim$20, and this drift accumulated each cycle until the "parallel stripes" became visibly non-parallel. We replaced the bang-bang turn with a proportional controller, $\omega = \text{clamp}(K_P \cdot \text{yaw\_err}, \pm\texttt{TURN\_SPEED})$: the command runs at maximum speed far from the target and ramps smoothly to zero near it, so the angular velocity is already small when the state transitions out and the overshoot becomes negligible. The same trick on the SHIFT distance produces clean stripe spacing.

Stuck handling. The original SHIFT exit conditions (full body width covered or front cone tripped) did not catch all stuck situations. We added a wall-clock timeout on SHIFT: if `BODY_WIDTH` has not been covered within `SHIFT_TIMEOUT` seconds, the SHIFT exits anyway and the U-turn completes.

Limitations. As discussed above, the boustrophedon strategy does not detect when the parallel wall on the opposite side of the room has been reached, so the robot keeps zig-zagging in the last stripe until it is stopped manually. A natural extension would be to detect that several consecutive SHIFT phases failed to make significant progress and either declare coverage complete (stop), reverse the lateral direction to march back, or hand off to a map-based planner.

A demonstration video is available at https://youtu.be/eYblzjBrKoo.

2 Task 2 – Behaviors of Robot Swarms

2.1 Overview

We implemented and analyzed a decentralized **swarm aggregation** behavior using the **ARGoS 3** simulator and a two-state **Lua** controller. A swarm of `foot-bot` robots operates in a closed 4 m × 4 m arena. By following a simple local rule – stop and wait whenever a neighbor is detected – a single stable cluster emerges as a global property of the swarm. We analyzed how the speed and stability of cluster formation depend on swarm size for $N \in \{5, 10, 20, 30, 50\}$. The full source, configuration, and analysis scripts live under `Task 2/`.

2.2 Implementation

Two-state FSM (`task2.lua`). Each robot runs the same decentralized controller with two states:

- **WANDER** – drive forward at `MAX_VELOCITY = 20 cm/s`, steering away from walls and other robots using the 24 infrared proximity sensors. At every step, query the Range-and-Bearing (RAB) sensor; if any other robot is detected within `STOP_DISTANCE = 25 cm`, transition to **STOPPED** and initialize a wait timer. LED indicator: green.
- **STOPPED** – set wheel velocities to $(0, 0)$ and decrement the `WAIT_TICKS = 30` ($= 3.0$ s) timer. When the timer reaches zero, return to **WANDER**. LED indicator: red.

Obstacle avoidance. Avoidance is computed as a virtual repulsion vector summed over the 24 proximity readings:

$$\mathbf{F}_{\text{repulsion}} = -\sum_{i=1}^{24} \big(v_i \cos \alpha_i \, \hat{\mathbf{x}} + v_i \sin \alpha_i \, \hat{\mathbf{y}} \big),$$

where $v_i \in [0, 1]$ is the proximity reading and α_i is the sensor angle. When $\|\mathbf{F}_{\text{repulsion}}\| < 0.05$ the robot drives straight; otherwise the differential wheel speeds are adjusted toward the direction of $\mathbf{F}_{\text{repulsion}}$ using `TURN_GAIN = 2.0`, clamped to $[-20, 20]$ cm/s.

Experiment pipeline. `task2.argos` defines the arena, the swarm population, sensors, and loop functions that log the per-tick fraction of stopped robots and per-robot neighbor counts to CSV. `run_experiments.py` patches the swarm size in a temporary copy of the configuration, runs `argos3` headlessly (`-z`) for 300 s at 10 ticks/s for each $N \in \{5, 10, 20, 30, 50\}$, and emits per-run and aggregated CSVs under `data/`. `plot_results.py` consumes the dataset and writes the three figures discussed below.

A cluster is defined as successfully formed when **99 % of the swarm** is simultaneously in the `STOPPED` state.

2.3 Subtask a – Stop on Neighbor Detection & Subtask b – Bounded Wait Time

Both subtasks are realized by the FSM above. The local rule ("stop on neighbor detection") corresponds to the `WANDER` → `STOPPED` transition triggered by the RAB sensor at 25 cm. The bounded wait time is the 3.0 s timer attached to the `STOPPED` state, after which the robot resumes wandering.

2.4 Subtask c – Tuning for Aggregation: Cluster Kinetics

Figure 2 shows the fraction of stopped robots over time for the five swarm sizes.

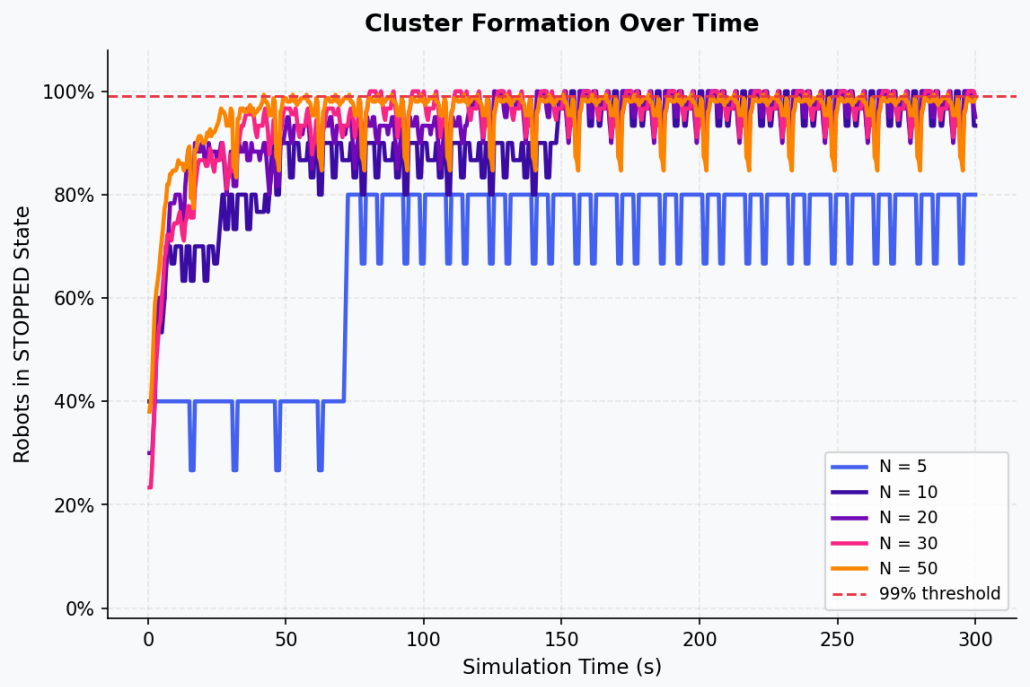

Figure 2: Fraction of stopped robots over time for each swarm size. A cluster is declared formed when the curve reaches 99 %.

Observations.

- $N = 50$ **(dense):** extremely rapid aggregation; the stopped fraction rises sharply and stabilizes at 100 % within ~ 50 s.
- $N = 30, 20, 10$ **(medium):** steady, reliable aggregation, reaching the 99 % threshold progressively later but remaining stable thereafter.
- $N = 5$ **(sparse):** fails to aggregate – the stopped fraction oscillates randomly and never reaches 99 %. Encounter rates are too low to sustain a growing cluster in this arena size.

2.5 Subtask d – Effect of Swarm Size on Speed and Stability

Time to cluster. Figure 3 and Table 1 summarize the time required for the swarm to reach the 99 % threshold.

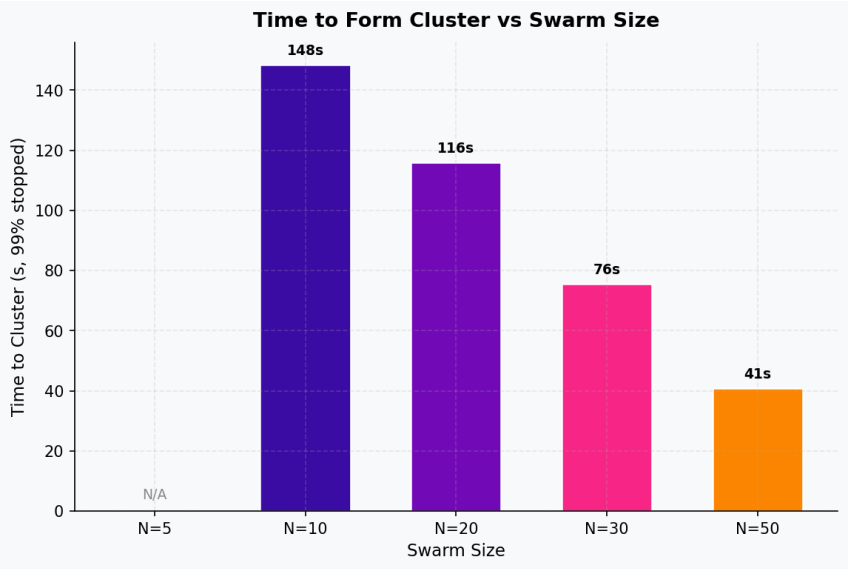

Figure 3: Time to form a cluster (99 % stopped) vs. swarm size.

Swarm size N	**Time to cluster (s)**
5	did not cluster
10	148.5
20	116.0
30	75.5
50	41.0

Table 1: Time required for the stopped fraction to reach 99 %.

The relationship is markedly non-linear: increasing N reduces the time to cluster super-linearly. Because the arena is fixed, a larger N corresponds directly to higher density, which raises the encounter rate. Once a few robots have stopped, they act as "nucleation sites" that wandering robots collide with, stop next to, and so grow the cluster – a positive-feedback loop that runs much faster at high density.

Cluster stability. Figure 4 and Table 2 report the distribution of per-robot neighbor counts in the last 3.0 s of each run.

N	**Mean**	**Median**	**Min**	**Max**
5	0.80	1	0	1
10	1.40	1	1	2
20	1.40	1	1	3
30	1.40	1	1	3
50	1.44	1	1	3

Table 2: Final-window neighbor-count statistics per swarm size.

For all successful runs ($N \geq 10$), the median neighbor count is 1 with a tight $[1, 3]$ spread, indicating a stable, tight aggregate. For $N = 5$ the minimum drops to 0: robots frequently

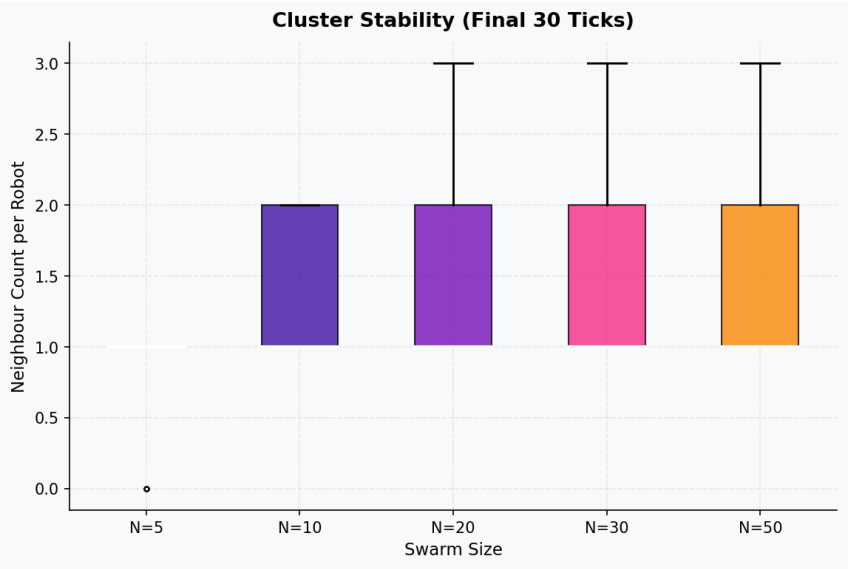

Figure 4: Distribution of neighbor counts per robot in the final 30 ticks of the simulation.

drift apart faster than new neighbors arrive to reinforce the stopping behavior, and the cluster dissolves.

2.6 Discussion

The system illustrates how a complex global structure (a single cluster) arises from a simple, local rule ("stop for a few seconds if anyone is nearby") with no central control, no global localization, and no communication beyond proximity detection. The mechanism is positive feedback: stopped robots act as physical obstacles that other robots steer toward and then stop next to themselves.

A critical density threshold ($N \geq 10$ in this arena) is needed for the positive-feedback loop to outrun the bounded wait time – below it, stopped robots time out and resume wandering before new neighbors arrive, and no cluster forms. Above the threshold, the time to cluster scales super-linearly with swarm size.

3 Completed Tasks

Task	Description	Status
1	Collision avoidance behavior (FSM, ROS 2 + Gazebo)	✓
1	Wall follower (PD + corner FSM, ROS 2 + Gazebo)	✓
1	Vacuum cleaner (boustrophedon coverage, ROS 2 + Gazebo)	✓
2	Stop on neighbor detection (ARGoS, Lua FSM)	✓
2	Bounded wait time before resuming wander	✓
2	Parameter tuning to obtain a single aggregated cluster	✓
2	Analysis of swarm size vs. speed and stability of cluster	✓